

GatoTeca

Proyecto de soluciones con SQL y PL/SQL para biblioteca.

Asignatura: Taller de Base de Datos
Sección: 004D
Docente: Carlos Orellana
Integrantes: Aolani Caiguan, Felipe Barra, Isidora Diaz, Renata Orellana

DuocUC

1. Análisis del contexto **3**
1.1 Descripción del proyecto 3
1.2 Alcance 3
1.3 Tecnologías utilizadas 4
1.4 Datos a Procesar e información relevante 4
1.4.1 Datos transaccionales 4
1.4.2 Datos de referencia 5
1.4.3 Objetos de Apoyo 6
1.5 Información relevante a generar 6
1.6 Reglas de Negocio 6
2. Desarrollo de procedimientos y funciones **7**
2.1 Definición y características 7
2.2 Ventajas y desventajas de cada uno 7
2.2.1 Ventajas 7
2.2.2 Desventajas 7
2.3 Propósito de procedimientos y funciones en el contexto del proyecto 8
2.4 Utilidad de procedimientos y funciones en el contexto del proyecto 9
2.4.1. Funciones usadas en sentencias SQL 9
2.4.2 Procedimientos ejecutados con EXEC 9
2.4.3. Reutilización dentro de otros programas PL/SQL 9
3. Desarrollo de Packages **10**
3.1 Definición y características 10
3.2 Ventajas y desventajas de utilizar packages 10
3.2.1 Ventajas 10
3.2.2 Desventajas 10
3.3 Cómo ayudan a organizar el código en el proyecto 10
3.4 Cómo se relacionan con los procedimientos y funciones 11
3.5 Cómo se utilizaron los packages en el proyecto 12
4. Desarrollo de Triggers **13**
4.1 Definición y características 13
4.2 Ventajas y desventajas en la utilización de triggers 13
4.3 Propósito de los triggers en el contexto del proyecto 13
4.4 Utilidad de los triggers en el contexto del proyecto 14
4.4.1 Trigger a nivel Fila 14
4.4.2 Trigger a nivel Setencia 15
4.5 Riesgos y Limitaciones 15
5. Proceso de desarrollo con IA **16**
5.1 Evidencia 1: decisión técnica 16
5.2 Evidencia 2: error o dificultad 16
5.3 Evidencia 3: prueba significativa 17
5.4 Evidencia 4: aprendizaje o cambio relevante 17
6. Conclusión **18**
6.1 Resumen 18
6.2 Impacto en el proyecto 18
6.3 Recomendaciones 19
6.4 Reflexión sobre el uso de IA 19
7. Anexos **20**
7.1 Código completo 20
7.1.1 Creación de Usuario 20
7.1.2 Creación de Tablas y Insercion de Datos 20
7.1.3 Objetos de Apoyo 30
7.1.4 Package 31
7.1.5 Triggers 38
7.1.6 Pruebas 40
7.2 Modelo MER 43

1. Análisis del contexto

1.1 Descripción del proyecto

GatoTeca es una biblioteca que combina el préstamo de libros con la convivencia de gatos que viven en el establecimiento. Además de pedir libros, los socios pueden alimentar a los gatos, siempre bajo la supervisión de un empleado.

El objetivo del proyecto es automatizar los procesos de la biblioteca: el control de préstamos y devoluciones, el cálculo de multas por atraso y el registro de la alimentación de los gatos.

Para lograrlo, se desarrolló una base de datos en Oracle normalizada en tercera forma normal (3FN) y un conjunto de objetos PL/SQL agrupados en el package `PKG_GATOTECA`:

- funciones para los cálculos del negocio (días de atraso, multas y montos);
- procedimientos para registrar préstamos, devoluciones y alimentaciones, y para procesar multas de forma masiva;
- triggers que garantizan que las reglas de negocio se cumplan aunque los datos se modifiquen fuera del package.

1.2 Alcance

Los programas PL/SQL buscan mejorar los procesos del negocio que tienen relación directa con la gestión de préstamos y con los gatos.

1. Gestión de préstamos: automatiza el registro de préstamos y devoluciones, valida que el ejemplar esté disponible antes de prestarlo y actualiza su estado en cada movimiento.
2. Control de multas: automatiza el cálculo y registro de multas por atraso, sin intervención manual, tanto en la revisión masiva de préstamos vencidos como al momento de la devolución.
3. Servicio de alimentación: automatiza el registro de cada alimentación, valida que la cantidad de porciones esté dentro de lo permitido y calcula el monto a pagar.
4. Auditoría: deja un registro automático de cada operación realizada sobre la tabla de multas.

Quedan fuera del alcance de esta entrega los reportes de recaudación por periodo y la gestión de pagos de multas.

1.3 Tecnologías utilizadas

1. Oracle Database: Es el motor de base de datos donde creamos tanto las tablas como los objetos en PL/SQL.
2. SQL: Lenguaje utilizado para la creación de tablas, límites del programa, DML, y consultas.
3. Visual Studio Code con la extensión de Oracle SQL: Editor desde el que se conecta la base de datos, ejecución de scripts y pruebas de código.
4. Git: Control de versiones de los scripts y documentación, facilitando el trabajo colaborativo.

1.4 Datos a Procesar e información relevante

El modelo de datos tiene 14 tablas de negocio, divididas en dos grupos según cómo se procesan. Además, cuenta con una tabla de apoyo para auditoría y tres secuencias.

1.4.1 Datos transaccionales

Son datos que se originan en las tareas del día a día del establecimiento. Crecen diariamente y se recorren por bloques para calcular deudas, atrasos y recaudaciones.

Entidad	Atributos
PRESTAMO	id_prestamo (PK), id_socio (FK), id_ejemplar (FK), id_empleado (FK), fecha_prestamo, fecha_devolucion_estimada, fecha_devolucion_real
MULTA	id_multa (PK), id_prestamo (FK), monto, pagada
ALIMENTACION	id_alimentacion (PK), id_socio (FK), id_gato (FK), id_alimento (FK), id_empleado (FK), fecha, cantidad
EJEMPLAR	id_ejemplar (PK), id_libro (FK), estado, ubicacion

1.4.2 Datos de referencia

Cambian con poca frecuencia y se cruzan con los datos transaccionales para darle el contexto en cálculos y reportes.

Entidad	Atributos
LIBRO	id_libro (PK), isbn, titulo, anio_publicacion, id_editorial (FK), id_categoria (FK)
AUTOR	id_autor (PK), nombre, apellido, nacionalidad
LIBRO_AUTOR	id_libro_autor (PK), id_libro (FK), id_autor (FK)
EDITORIAL	id_editorial (PK), nombre, pais
CATEGORIA	id_categoria (PK), nombre
SOCIO	id_socio (PK), rut, nombre, apellido, email, telefono, direccion
EMPLEADO	id_empleado (PK), rut, nombre, apellido, cargo
GATO	id_gato (PK), nombre, sexo, color, fecha_nacimiento, fecha_llegada, id_raza (FK)
RAZA	id_raza (PK), nombre
ALIMENTO	id_alimento (PK), nombre, tipo, precio

1.4.3 Objetos de Apoyo

Objeto	Uso
AUD_GATOTECA	Tabla de auditoría (id_aud, tabla, operacion, usuario, fecha). Llena el trigger de sentencia.
SEQ_PRESTAMO, SEQ_MULTA, SEQ_ALIMENTACION	Generan las PK de los registros que crea el package.

1.5 Información relevante a generar

Con el código PL/SQL implementado, el sistema podrá generar la siguiente información:

1. Préstamos vencidos: listado con el socio, el libro y la multa acumulada, mediante una consulta que usa `pkg_gatoteca.fn_calcular_multa`.
2. Multas: el monto de cada préstamo atrasado, siempre con la misma regla ($500 por día), guardado en la tabla `MULTA`.
3. Estado de los ejemplares: siempre actualizado gracias a los triggers, lo que permite contar con una consulta simple de cuántos están disponibles, prestados, dañados o perdidos.
4. Alimentación: el monto a pagar por cada alimentación, que se informa al registrarla.
5. Gatos: el total de gatos registrados, con `pkg_gatoteca.fn_total_gatos_activos`.
6. Auditoría: el historial de operaciones sobre multas, con usuario y fecha.

1.6 Reglas de Negocio

1. Cada préstamo dura 7 días y corresponde a un solo ejemplar específico.
2. Un ejemplar prestado, dañado o perdido no se puede volver a prestar.
3. Si un socio devuelve un libro con retraso, se genera una multa de $500 por cada día de atraso.
4. En cada visita, el socio puede comprar de 1 a 5 porciones. El monto a pagar se obtiene multiplicando las porciones por el precio del alimento elegido.

2. Desarrollo de procedimientos y funciones

2.1 Definición y características

1. Procedimiento almacenado: Es un bloque PL/SQL con nombre, este se guarda en la base de datos y ejecuta una serie de acciones como consulta, inserción, actualización o muestra de datos. No devuelven un valor directamente a pesar de tener opcionalmente los parámetros.

2. Función almacenada: Es un bloque PL/SQL que ejecuta un cálculo y siempre devuelve un valor, en caso de no modificar datos se puede usar dentro de sentencias SQL y también desde otros programas PL/SQL.

2.2 Ventajas y desventajas de cada uno

2.2.1 Ventajas

1. Procedimientos: Centralizan procesos del negocio de forma completa, procesando grandes volúmenes de datos permitiendo manejar los errores con mensajes claros.
2. Funciones: Permiten reutilizar una misma ejecución de código en muchas partes del sistema, entonces se evita guardar los datos calculados, respetando la tercera forma normal, entonces las modificaciones se mantienen solo donde ocurre el cambio y no salen de ahí.

2.2.2 Desventajas

1. Procedimientos: No se pueden usar dentro de una sentencia SQL y en caso de tener mucha lógica puede ser difícil en cuanto a testeo y mantención.
2. Funciones: Usadas sobre muchas filas en una consulta pueden volverla más lenta, no se deben modificar los datos si se quieren usar en sentencias SQL.

2.3 Propósito de procedimientos y funciones en el contexto del proyecto

Todos los objetos están dentro del package `PKG_GATOTECA`. Los públicos se pueden usar desde fuera; los privados solo dentro del cuerpo del package.

Funciones:

1. `fn_calcular_multa` (pública, con parámetros): calcula la multa de un préstamo a $500 por día de atraso, para que siempre se cobre igual. Se puede usar en consultas SQL y dentro del package.
2. `fn_total_gatos_activos` (pública, sin parámetros): devuelve el total de gatos registrados. Como la tabla GATO no tiene columna de estado, cuenta todos los gatos.
3. `fn_dias_atraso` (privada, con parámetros): calcula cuántos días de atraso tiene un préstamo, esté devuelto o no. Si todavía no se devuelve, usa la fecha de hoy.
4. `fn_total_alimentacion` (privada, con parámetros): calcula el monto que debe pagar el socio según las porciones y el precio del alimento elegido.
5. `existe_socio`, `existe_empleado`, `existe_gato` y `existe_alimento` (privadas, con parámetros): validan que el registro exista antes de operar.
6. `estado_ejemplar` (privada, con parámetros): devuelve el estado del ejemplar y bloquea la fila para que dos usuarios no presten el mismo ejemplar al mismo tiempo.

Procedimientos

1. `sp_registrar_prestamo` (público, con parámetros): valida que el socio, el empleado y el ejemplar existan y que el ejemplar esté disponible. Después registra el préstamo con devolución a 7 días.
2. `sp_devolucion_ejemplar` (público, con parámetros): registra la devolución y, si hay atraso, crea o actualiza la multa.
3. `sp_registrar_alimentacion` (público, con parámetros): valida que las porciones están entre 1 y 5 y que los datos existan, registra la alimentación y muestra el monto a pagar.
4. `sp_generar_multas_masivo` (público, sin parámetros): procesamiento masivo que recorre todos los préstamos vencidos sin devolver y genera o actualiza sus multas.
5. `registrar_multa` (privado, con parámetros): crea la multa del préstamo o actualiza su monto si sigue sin pagar. Lo usan para la devolución y el proceso masivo.

2.4 Utilidad de procedimientos y funciones en el contexto del proyecto

2.4.1. Funciones usadas en sentencias SQL

```
SELECT pkg_gatoteca.fn_total_gatos_activos AS total_gatos FROM dual;
SELECT p.id_prestamo,
       s.nombre || ' ' || s.apellido                    AS socio,
       l.titulo                                         AS libro,
       TRUNC(SYSDATE) - TRUNC(p.fecha_devolucion_estimada) AS
dias_atraso,
       pkg_gatoteca.fn_calcular_multa(p.id_prestamo)    AS multa
  FROM prestamo p
  JOIN socio    s ON s.id_socio    = p.id_socio
  JOIN ejemplar e ON e.id_ejemplar = p.id_ejemplar
  JOIN libro    l ON l.id_libro    = e.id_libro
 WHERE p.fecha_devolucion_real IS NULL
   AND TRUNC(p.fecha_devolucion_estimada) < TRUNC(SYSDATE);
```

`fn_dias_atraso` no se puede llamar desde SQL porque es privada; por eso en la consulta los días se calculan con las fechas.

2.4.2 Procedimientos ejecutados con EXEC

```
EXEC pkg_gatoteca.sp_registrar_prestamo(1, 2, 1);           -- socio,
ejemplar, empleado
EXEC pkg_gatoteca.sp_devolucion_ejemplar(12);
EXEC pkg_gatoteca.sp_registrar_alimentacion(1, 1, 1, 1, 2); -- socio,
gato, alimento, empleado, porciones

EXEC pkg_gatoteca.sp_generar_multas_masivo;                 -- sin
parámetros
```

2.4.3. Reutilización dentro de otros programas PL/SQL

- `sp_generar_multas_masivo` y `sp_devolucion_ejemplar` usan `fn_calcular_multa` y `registrar_multa`.
- `fn_calcular_multa` usa `fn_dias_atraso`.
- `sp_registrar_alimentacion` usa `fn_total_alimentacion`.

Así la regla de los $500 por día existe en un solo lugar: la constante `c_multa_por_dia`.

3. Desarrollo de Packages

3.1 Definición y características

1. Un package es un objeto que agrupa los procedimientos, funciones, variables y constantes relacionadas bajo una misma expresión.
2. Posee una especificación, es decir, declara los elementos públicos utilizables desde afuera y un cuerpo que contiene el código y los elementos privados de uso interno.

3.2 Ventajas y desventajas de utilizar packages

3.2.1 Ventajas

1. Orden: agrupa el código por área del negocio.
2. Encapsulamiento: los elementos privados no se pueden llamar desde fuera, así nadie puede saltarse las validaciones.
3. Rendimiento: el package completo se carga en memoria la primera vez que se usa, y las llamadas siguientes son más rápidas.

3.2.2 Desventajas

1. Si se cambia la especificación, hay que recompilar los objetos que dependen del package.
2. Un package muy grande puede volverse difícil de mantener.

3.3 Cómo ayudan a organizar el código en el proyecto

1. Se creó un único package, `PKG_GATOTECA`, que reúne en un solo lugar las operaciones diarias del negocio: préstamos, devoluciones, multas y alimentación de gatos.
2. En la especificación se declararon solo los elementos que se usan desde fuera:
    a. cuatro procedimientos: `sp_registrar_prestamo`, `sp_devolucion_ejemplar`, `sp_registrar_alimentacion` y `sp_generar_multas_masivo`;
    b. dos funciones: `fn_calcular_multa` y `fn_total_gatos_activos`.

3. En el cuerpo quedaron privados:
    a. las validaciones: `existe_socio`, `existe_empleado`, `existe_gato`, `existe_alimento` y `estado_ejemplar`;
    b. los cálculos internos: `fn_dias_atraso` y `fn_total_alimentacion`;
    c. el procedimiento `registrar_multa`;
    d. las constantes del negocio: 7 días de préstamo, mínimo 1 y máximo 5 porciones, $500 por día y el estado `DISPONIBLE`.

Así se protegen las reglas de negocio y se evita un mal uso desde fuera.

3.4 Cómo se relacionan con los procedimientos y funciones

El package no depende de funciones externas: todas las funciones están dentro de él, y los procedimientos públicos se apoyan en las funciones y procedimientos privados del cuerpo.

- `sp_devolucion_ejemplar` usa `fn_dias_atraso` para saber si hay atraso, `fn_calcular_multa` para calcular el monto y `registrar_multa` para guardarla.
- `sp_generar_multas_masivo` reutiliza `fn_calcular_multa` y `registrar_multa`, igual que la devolución.
- `sp_registrar_alimentacion` usa `fn_total_alimentacion` para calcular el monto final.
- `sp_registrar_prestamo` usa las validaciones `existe_*` y `estado_ejemplar`.

`fn_calcular_multa` y `fn_total_gatos_activos` se dejaron públicas porque también se necesitan en consultas SELECT. Como solo leen datos, Oracle permite usarlas dentro de sentencias SQL.

3.5 Cómo se utilizaron los packages en el proyecto

1. `sp_registrar_prestamo(socio, ejemplar, empleado)`: valida que el socio y el empleado existan y que el ejemplar esté DISPONIBLE. Si todo está bien, registra el préstamo con fecha de devolución a 7 días, y el trigger deja el ejemplar como PRESTADO.
2. `sp_devolucion_ejemplar(prestamo)`: valida que el préstamo exista y no haya sido devuelto, y registra la fecha de devolución. El trigger deja el ejemplar DISPONIBLE. Si hay atraso, crea la multa o actualiza la que ya existía, porque cada préstamo puede tener una sola.
3. `sp_registrar_alimentacion(socio, gato, alimento, empleado, porciones)`: valida que las porciones estén entre 1 y 5 y que los datos existan, registra la alimentación y muestra el monto a pagar.
4. `sp_generar_multas_masivo`: recorre todos los préstamos vencidos sin devolver y genera o actualiza sus multas. Se puede ejecutar varias veces sin duplicar multas.
5. Manejo de errores: cada procedimiento valida los datos y, si algo no se cumple, lanza un error personalizado con `RAISE_APPLICATION_ERROR`. Por ejemplo, al pedir 7 porciones aparece ORA-20001: La cantidad de porciones excede el límite permitido por visita (Máx: 5). Cuando ocurre un error, Oracle deshace automáticamente lo que hizo esa llamada. El package no ejecuta `COMMIT` ni `ROLLBACK`: confirmar o deshacer la transacción le corresponde a quien lo invoca. Así una operación del package se puede combinar con otras dentro de la misma transacción.

Código	Situación
-20001 / -20002	Más de 5 / menos de 1 porción
-20003 a -20006	Socio, gato, alimento o empleado inexistente (alimentación)
-20010 a -20013	Socio, empleado o ejemplar inexistente, o ejemplar no disponible (préstamo)
-20020 / -20021	Préstamo inexistente / ya devuelto

4. Desarrollo de Triggers

4.1 Definición y características

1. Un trigger es un bloque PL/SQL asociado a una tabla que se ejecuta automáticamente cuando ocurre un evento sobre ella (INSERT, UPDATE o DELETE), ya sea antes (BEFORE) o después (AFTER) del evento.
2. Trigger de sentencia: se ejecuta una sola vez por cada instrucción, sin importar cuántas filas afecte.
3. Trigger de fila (FOR EACH ROW): se ejecuta una vez por cada fila afectada y puede leer los valores anteriores (:OLD) y nuevos (:NEW) de esa fila.

4.2 Ventajas y desventajas en la utilización de triggers

1. Ventajas: garantizan que las reglas del negocio se cumplan siempre, aunque los datos se modifiquen sin pasar por los procedimientos, y automatizan tareas como actualizar estados o registrar multas.
2. Desventajas: actúan de forma "invisible", lo que puede dificultar encontrar errores. Además, no reaccionan al paso del tiempo, y un trigger de fila no puede consultar la misma tabla que lo dispara (error de tabla mutante, ORA-04091).

4.3 Propósito de los triggers en el contexto del proyecto

En GatoTeca existen reglas que no se pueden saltar: no se pueden prestar ejemplares que no estén disponibles, mantener sincronizado de forma automática el estado de cada ejemplar en caso de ser prestado o devuelto, llevar un registro de auditoría sobre las modificaciones hechas sobre las multas. Los triggers tienen como fin ser la última barrera de seguridad, aunque alguien inserte o actualice datos directamente en una tabla sin pasar por el package `PKG_GATOTECA`, las reglas del negocio y la trazabilidad se cumplen igualmente.

Como los triggers no reaccionan al paso del tiempo por sí solos, las multas por atraso se gestionan desde el package:

1. Devolución: `PKG_GATOTECA.sp_devolucion_ejemplar` calcula los días de atraso y registra o actualiza la multa
2. Revisiones masivas: `PKG_GATOTECA.sp_generar_multas_masivo` recorre los préstamos activos vencidos y genera o actualiza sus multas.

4.4 Utilidad de los triggers en el contexto del proyecto

4.4.1 Trigger a nivel Fila

1. `trg_prestamo_bir` (`BEFORE INSERT` sobre `PRESTAMO`): si alguien intenta insertar un préstamo directamente en la tabla, sin usar `PKG_GATOTECA.sp_registrar_prestamo`, este trigger consulta el estado actual del ejemplar.
    a. Si el ejemplar no está `'DISPONIBLE'`, rechaza la inserción con el error personalizado `ORA-20030`.
    b. Si el ejemplar no existe, arroja el error `ORA-20032`.
2. `trg_prestamo_estado_ejemplar` (`AFTER INSERT OR UPDATE OF fecha_devolucion_real` sobre `PRESTAMO`): automatiza el cambio de estado en la tabla `EJEMPLAR`.
    a. Cuando se inserta un préstamo sin fecha de devolución real, deja el ejemplar como `'PRESTADO'`.
    b. Cuando se registra la `fecha_devolucion_real`, lo vuelve a dejar `'DISPONIBLE'`.

Como solo usa `:NEW` y `:OLD` y modifica únicamente `EJEMPLAR`, sin consultar `PRESTAMO`, no provoca el error de tabla mutante (`ORA-04091`).

4.4.2 Trigger a nivel Setencia

1. `trg_multa_aud_stmt` (`AFTER INSERT OR UPDATE OR DELETE` sobre `MULTA`): lleva la trazabilidad y auditoría de la tabla de multas. Se ejecuta una sola vez por cada instrucción DML, sin importar cuántas filas afecte (por ejemplo, durante el procesamiento masivo de multas). En cada ejecución inserta un registro en la tabla de apoyo `AUD_GATOTECA` con la tabla afectada (`'MULTA'`), la operación realizada (`'INSERT'`, `'UPDATE'` o `'DELETE'`), el usuario y la fecha.

4.5 Riesgos y Limitaciones

1. La validación de disponibilidad está tanto en el package como en `trg_prestamo_bir`. Es redundante a propósito, como segunda barrera, pero si la regla cambia hay que modificarla en los dos lugares.
2. `trg_multa_aud_stmt` registra qué operación se hizo, pero no qué filas cambiaron ni sus valores anteriores. Para eso haría falta un trigger de fila.
3. Los triggers no reaccionan al paso del tiempo: si nadie ejecuta `sp_generar_multas_masivo`, las multas de los préstamos vencidos no se actualizan.
4. Si alguien cambia a mano el estado de un ejemplar (por ejemplo a DANADO o PERDIDO), ningún trigger valida ese cambio.

5. Proceso de desarrollo con IA

Se usó Claude para revisar el informe contra el código del repositorio y las instrucciones del docente, y para corregir la documentación. La IA leyó el código, pero no lo ejecutó en Oracle: cada hallazgo se verificó contra los archivos del repositorio y, cuando correspondía, fue ejecutado en la base de datos.

5.1 Evidencia 1: decisión técnica

- Problema: el ejemplo de la sección 2.4 usaba `pkg_gatoteca.fn_dias_atraso` dentro de un SELECT. Esa función es privada, porque está solo en el cuerpo del package.
- Alternativas consideradas:
  - hacer pública `fn_dias_atraso`, agregándola a la especificación;
  - mantenerla privada y calcular los días en la consulta directamente con las fechas.
- Solución elegida: mantenerla privada. La consulta calcula `TRUNC(SYSDATE) - TRUNC(p.fecha_devolucion_estimada)` y usa la función pública `fn_calcular_multa` para el monto.
- Por qué: lo que se necesita fuera del package es el monto de la multa, no los días. Hacerla pública solo para un ejemplo rompía el encapsulamiento que se había diseñado.

5.2 Evidencia 2: error o dificultad

- Problema: al revisar el informe contra el repositorio, la IA encontró que varias secciones describían un diseño anterior que ya no existía en el código:
  - la 3.4 decía que las funciones estaban fuera del package;
  - la 3.3 decía que la especificación tenía solo tres procedimientos, cuando tiene cuatro procedimientos y dos funciones;
  - la 3.5 decía que cada procedimiento hacía `COMMIT` o `ROLLBACK`, y el package no tiene ninguno;
  - la 1.5 prometía reportes de recaudación que ningún objeto genera.
- Cómo se detectó: comparando cada afirmación del informe con `pkg_gatoteca.sql` y `tgr_gatoteca.sql`.
- Cómo se resolvió:
  - se reescribieron las secciones 1.5, 2.3, 2.4, 3.3, 3.4 y 3.5 según el código real;
  - en el manejo de errores se explicó que, ante un `RAISE_APPLICATION_ERROR`, Oracle deshace lo que hizo esa llamada, y que confirmar la transacción le corresponde a quien invoca el package.

5.3 Evidencia 3: prueba significativa

- Qué se quería comprobar: que el código incluido en el anexo del informe fuera exactamente el mismo que está en el repositorio, para asegurar que lo documentado corresponde a lo que realmente se entrega.
- Datos y condiciones: comparación automática (`diff`) entre el código del anexo y los archivos `packages/pkg_gatoteca.sql` y `triggers/tgr_gatoteca.sql` del repositorio.
- Resultado esperado: código idéntico.
- Resultado obtenido: el código del anexo era idéntico al del repositorio; solo cambiaba la sangría.
- Conclusión: el código estaba bien. Los errores estaban en el texto que lo describía (secciones 1.5, 2.4, 3.3, 3.4 y 3.5), y por eso se corrigió el texto y no el código.

5.4 Evidencia 4: aprendizaje o cambio relevante

- Qué cambió: el proyecto pasó de dos packages (`pkg_prestamos` y `pkg_gatos`), funciones sueltas y un trigger de horario a un único `PKG_GATOTECA`. El código se actualizó, pero el informe y el README quedaron describiendo la versión anterior.
- Qué comprendimos mejor: que la documentación también hay que verificarla contra el código, y que un texto puede sonar correcto y aun así estar equivocado. También comprendimos mejor qué significa que un elemento sea privado: no se puede usar ni siquiera desde una consulta SQL.
- Qué haríamos distinto: actualizar el informe y el README en el mismo commit en que cambia el diseño del código, y no dejarlo para el final.

6. Conclusión

6.1 Resumen

Se desarrolló una base de datos normalizada en 3FN, con 14 tablas de negocio, para gestionar el catálogo de libros, los préstamos y el servicio de alimentación de gatos. Sobre ella se integró PL/SQL:

- funciones para los cálculos;
- procedimientos con y sin parámetros para las operaciones diarias y el procesamiento masivo de multas;
- el package `PKG_GATOTECA`, que agrupa la lógica de negocio separando lo público de lo privado;
- triggers de fila y de sentencia que aseguran la integridad de los datos y la auditoría de las multas.

6.2 Impacto en el proyecto

La biblioteca reduce de forma significativa el error humano y ahorra tiempo en tareas rutinarias, como el cálculo de multas y el registro de préstamos y alimentaciones. Como los montos se calculan con funciones en vez de escribirse a mano, la regla de cobro es siempre la misma y la información se mantiene íntegra y confiable para la toma de decisiones.

6.3 Recomendaciones

1. Programar `sp_generar_multas_masivo` con `DBMS_SCHEDULER` para que se ejecute todos los días sin intervención manual.
2. Agregar un procedimiento para pagar multas (marcar `pagada = 'S'`) y bloquear nuevos préstamos a socios con multas impagas.
3. Crear un procedimiento de reportes de recaudación de alimentación y multas por periodo.
4. Agregar una columna de estado a `GATO` (activo, adoptado) para que `fn_total_gatos_activos` cuente solo los gatos activos.
5. Ampliar la auditoría con triggers de fila que guarden los valores anteriores y nuevos.

6.4 Reflexión sobre el uso de IA

La IA fue útil para revisar rápidamente el informe contra el código y la pauta, y para detectar contradicciones que se nos habían pasado. Sin embargo, no ejecutó el código: sus observaciones se basaban en leerlo, así que cada una tuvo que comprobarse en los archivos del repositorio o en Oracle. También aprendimos que la IA puede proponer textos completos, pero las evidencias de nuestro propio proceso solo podíamos aportarlas nosotros.

7. Anexos

7.1 Código completo

7.1.1 Creación de Usuario

```
ALTER SESSION SET "_ORACLE_SCRIPT"=true;

CREATE USER GatoTecaDeveloper IDENTIFIED BY "GatoTeca2026";

GRANT CONNECT, RESOURCE TO GatoTecaDeveloper;
GRANT CREATE VIEW, CREATE PROCEDURE, CREATE TRIGGER TO GatoTecaDeveloper;
ALTER USER GatoTecaDeveloper QUOTA UNLIMITED ON USERS;
```

7.1.2 Creación de Tablas y Insercion de Datos

```
-- ------------------------------------------------------
-- 0. LIMPIEZA (permite ejecutar el script varias veces)
-- ------------------------------------------------------
BEGIN
  FOR t IN (SELECT table_name FROM user_tables
            WHERE table_name IN
('ALIMENTACION','ALIMENTO','GATO','RAZA','MULTA','PRESTAMO','EJEMPLAR','LIB
RO_AUTOR','LIBRO',

'SOCIO','EMPLEADO','AUTOR','CATEGORIA','EDITORIAL')) LOOP
    EXECUTE IMMEDIATE 'DROP TABLE ' || t.table_name || ' CASCADE
CONSTRAINTS';
  END LOOP;
END;
/

-- ------------------------------------------------------
-- 1. TABLAS
-- ------------------------------------------------------
CREATE TABLE editorial (
  id_editorial  NUMBER(5)     CONSTRAINT pk_editorial PRIMARY KEY,
  nombre        VARCHAR2(80)  NOT NULL,
  pais          VARCHAR2(50)  NOT NULL,
  CONSTRAINT uq_editorial_nombre UNIQUE (nombre)
);

CREATE TABLE categoria (
```

```
  id_categoria  NUMBER(5)     CONSTRAINT pk_categoria PRIMARY KEY,
  nombre        VARCHAR2(50)  NOT NULL,
  CONSTRAINT uq_categoria_nombre UNIQUE (nombre)
);

CREATE TABLE autor (
  id_autor      NUMBER(5)     CONSTRAINT pk_autor PRIMARY KEY,
  nombre        VARCHAR2(50)  NOT NULL,
  apellido      VARCHAR2(50)  NOT NULL,
  nacionalidad  VARCHAR2(50)
);

CREATE TABLE libro (
  id_libro          NUMBER(6)     CONSTRAINT pk_libro PRIMARY KEY,
  isbn              VARCHAR2(13)  NOT NULL,
  titulo            VARCHAR2(150) NOT NULL,
  anio_publicacion  NUMBER(4),
  id_editorial      NUMBER(5)     NOT NULL,
  id_categoria      NUMBER(5)     NOT NULL,
  CONSTRAINT fk_libro_editorial FOREIGN KEY (id_editorial) REFERENCES
editorial(id_editorial),
  CONSTRAINT fk_libro_categoria FOREIGN KEY (id_categoria) REFERENCES
categoria(id_categoria),
  CONSTRAINT uq_libro_isbn UNIQUE (isbn),
  CONSTRAINT ck_libro_anio CHECK (anio_publicacion BETWEEN 1450 AND 2100)
);

CREATE TABLE libro_autor (
  id_libro_autor  NUMBER(6)  CONSTRAINT pk_libro_autor PRIMARY KEY,
  id_libro        NUMBER(6)  NOT NULL,
  id_autor        NUMBER(5)  NOT NULL,
  CONSTRAINT fk_la_libro FOREIGN KEY (id_libro) REFERENCES libro(id_libro),
  CONSTRAINT fk_la_autor FOREIGN KEY (id_autor) REFERENCES autor(id_autor),
  CONSTRAINT uq_libro_autor UNIQUE (id_libro, id_autor)
);

CREATE TABLE ejemplar (
  id_ejemplar  NUMBER(6)     CONSTRAINT pk_ejemplar PRIMARY KEY,
  id_libro     NUMBER(6)     NOT NULL,
  estado       VARCHAR2(12)  DEFAULT 'DISPONIBLE' NOT NULL,
  ubicacion    VARCHAR2(20),
  CONSTRAINT fk_ejemplar_libro FOREIGN KEY (id_libro) REFERENCES
libro(id_libro),
  CONSTRAINT ck_ejemplar_estado CHECK (estado IN
('DISPONIBLE','PRESTADO','DANADO','PERDIDO'))
);
```

```
CREATE TABLE socio (
  id_socio   NUMBER(6)     CONSTRAINT pk_socio PRIMARY KEY,
  rut        VARCHAR2(12)  NOT NULL,
  nombre     VARCHAR2(50)  NOT NULL,
  apellido   VARCHAR2(50)  NOT NULL,
  email      VARCHAR2(100),
  telefono   VARCHAR2(15),
  direccion  VARCHAR2(120),
  CONSTRAINT uq_socio_rut UNIQUE (rut)
);

CREATE TABLE empleado (
  id_empleado  NUMBER(5)     CONSTRAINT pk_empleado PRIMARY KEY,
  rut          VARCHAR2(12)  NOT NULL,
  nombre       VARCHAR2(50)  NOT NULL,
  apellido     VARCHAR2(50)  NOT NULL,
  cargo        VARCHAR2(40)  NOT NULL,
  CONSTRAINT uq_empleado_rut UNIQUE (rut)
);

CREATE TABLE prestamo (
  id_prestamo               NUMBER(8)  CONSTRAINT pk_prestamo PRIMARY KEY,
  id_socio                  NUMBER(6)  NOT NULL,
  id_ejemplar               NUMBER(6)  NOT NULL,
  id_empleado               NUMBER(5)  NOT NULL,
  fecha_prestamo            DATE       DEFAULT SYSDATE NOT NULL,
  fecha_devolucion_estimada DATE       NOT NULL,
  fecha_devolucion_real     DATE,
  CONSTRAINT fk_prestamo_socio    FOREIGN KEY (id_socio)    REFERENCES
socio(id_socio),
  CONSTRAINT fk_prestamo_ejemplar FOREIGN KEY (id_ejemplar) REFERENCES
ejemplar(id_ejemplar),
  CONSTRAINT fk_prestamo_empleado FOREIGN KEY (id_empleado) REFERENCES
empleado(id_empleado),
  CONSTRAINT ck_prestamo_fechas CHECK (fecha_devolucion_estimada >
fecha_prestamo)
);

CREATE TABLE multa (
  id_multa     NUMBER(8)  CONSTRAINT pk_multa PRIMARY KEY,
  id_prestamo  NUMBER(8)  NOT NULL,
  monto        NUMBER(8)  NOT NULL,
  pagada       CHAR(1)    DEFAULT 'N' NOT NULL,
  CONSTRAINT fk_multa_prestamo FOREIGN KEY (id_prestamo) REFERENCES
prestamo(id_prestamo),
```

```
  CONSTRAINT uq_multa_prestamo UNIQUE (id_prestamo),
  CONSTRAINT ck_multa_monto  CHECK (monto > 0),
  CONSTRAINT ck_multa_pagada CHECK (pagada IN ('S','N'))
);


-- ---------- GATOS ----------
CREATE TABLE raza (
  id_raza  NUMBER(3)     CONSTRAINT pk_raza PRIMARY KEY,
  nombre   VARCHAR2(50)  NOT NULL,
  CONSTRAINT uq_raza_nombre UNIQUE (nombre)
);

CREATE TABLE gato (
  id_gato           NUMBER(5)     CONSTRAINT pk_gato PRIMARY KEY,
  nombre            VARCHAR2(40)  NOT NULL,
  sexo              CHAR(1)       NOT NULL,
  color             VARCHAR2(40),
  fecha_nacimiento  DATE,
  fecha_llegada     DATE          NOT NULL,
  id_raza           NUMBER(3)     NOT NULL,
  CONSTRAINT fk_gato_raza FOREIGN KEY (id_raza) REFERENCES raza(id_raza),
  CONSTRAINT ck_gato_sexo CHECK (sexo IN ('M','H'))
);

-- Tipos de comida que vende la Gatoteca
CREATE TABLE alimento (
  id_alimento  NUMBER(4)     CONSTRAINT pk_alimento PRIMARY KEY,
  nombre       VARCHAR2(60)  NOT NULL,
  tipo         VARCHAR2(10)  NOT NULL,
  precio       NUMBER(6)     NOT NULL,
  CONSTRAINT uq_alimento_nombre UNIQUE (nombre),
  CONSTRAINT ck_alimento_tipo   CHECK (tipo IN ('SECO','HUMEDO','SNACK')),
  CONSTRAINT ck_alimento_precio CHECK (precio > 0)
);

-- Cada vez que un socio paga por darle comida a un gato
CREATE TABLE alimentacion (
  id_alimentacion  NUMBER(8)  CONSTRAINT pk_alimentacion PRIMARY KEY,
  id_socio         NUMBER(6)  NOT NULL,
  id_gato          NUMBER(5)  NOT NULL,
  id_alimento      NUMBER(4)  NOT NULL,
  id_empleado      NUMBER(5)  NOT NULL,
  fecha            DATE       DEFAULT SYSDATE NOT NULL,
  cantidad         NUMBER(2)  NOT NULL,
```

```
  CONSTRAINT fk_alim_socio    FOREIGN KEY (id_socio)    REFERENCES
socio(id_socio),
  CONSTRAINT fk_alim_gato     FOREIGN KEY (id_gato)     REFERENCES
gato(id_gato),
  CONSTRAINT fk_alim_alimento FOREIGN KEY (id_alimento) REFERENCES
alimento(id_alimento),
  CONSTRAINT fk_alim_empleado FOREIGN KEY (id_empleado) REFERENCES
empleado(id_empleado),
  CONSTRAINT ck_alim_cantidad CHECK (cantidad BETWEEN 1 AND 5)
);

-- ------------------------------------------------------
-- 2. DATOS DE PRUEBA (minimo 10 por tabla)
-- ------------------------------------------------------

-- EDITORIAL (ids 1..10)
INSERT INTO editorial VALUES (1, 'Sudamericana', 'Argentina');
INSERT INTO editorial VALUES (2, 'Penguin Random House', 'Estados Unidos');
INSERT INTO editorial VALUES (3, 'Editorial Universitaria', 'Chile');
INSERT INTO editorial VALUES (4, 'Planeta', 'Espana');
INSERT INTO editorial VALUES (5, 'Alfaguara', 'Espana');
INSERT INTO editorial VALUES (6, 'Anagrama', 'Espana');
INSERT INTO editorial VALUES (7, 'Zig-Zag', 'Chile');
INSERT INTO editorial VALUES (8, 'LOM Ediciones', 'Chile');
INSERT INTO editorial VALUES (9, 'Pearson', 'Reino Unido');
INSERT INTO editorial VALUES (10, 'O''Reilly Media', 'Estados Unidos');

-- CATEGORIA (ids 1..10)
INSERT INTO categoria VALUES (1, 'Novela');
INSERT INTO categoria VALUES (2, 'Poesia');
INSERT INTO categoria VALUES (3, 'Ciencia Ficcion');
INSERT INTO categoria VALUES (4, 'Informatica');
INSERT INTO categoria VALUES (5, 'Historia');
INSERT INTO categoria VALUES (6, 'Fantasia');
INSERT INTO categoria VALUES (7, 'Cuento');
INSERT INTO categoria VALUES (8, 'Ensayo');
INSERT INTO categoria VALUES (9, 'Terror');
INSERT INTO categoria VALUES (10, 'Infantil');

-- AUTOR (ids 1..13)
INSERT INTO autor VALUES (1, 'Gabriel', 'Garcia Marquez', 'Colombiana');
INSERT INTO autor VALUES (2, 'Isabel', 'Allende', 'Chilena');
INSERT INTO autor VALUES (3, 'Pablo', 'Neruda', 'Chilena');
INSERT INTO autor VALUES (4, 'Isaac', 'Asimov', 'Estadounidense');
INSERT INTO autor VALUES (5, 'Robert', 'Martin', 'Estadounidense');
INSERT INTO autor VALUES (6, 'Micah', 'Martin', 'Estadounidense');
```

```
INSERT INTO autor VALUES (7, 'Gabriela', 'Mistral', 'Chilena');
INSERT INTO autor VALUES (8, 'Julio', 'Cortazar', 'Argentina');
INSERT INTO autor VALUES (9, 'Stephen', 'King', 'Estadounidense');
INSERT INTO autor VALUES (10, 'J.R.R.', 'Tolkien', 'Britanica');
INSERT INTO autor VALUES (11, 'Jorge Luis', 'Borges', 'Argentina');
INSERT INTO autor VALUES (12, 'Antoine', 'de Saint-Exupery', 'Francesa');
INSERT INTO autor VALUES (13, 'Yuval Noah', 'Harari', 'Israeli');

-- LIBRO (13 libros)
INSERT INTO libro VALUES (1, '9780307474728', 'Cien anos de soledad',
1967, 1, 1);
INSERT INTO libro VALUES (2, '9780525433477', 'La casa de los espiritus',
1982, 2, 1);
INSERT INTO libro VALUES (3, '9789561119270', 'Veinte poemas de amor',
1924, 3, 2);
INSERT INTO libro VALUES (4, '9780553293357', 'Fundacion',
1951, 2, 3);
INSERT INTO libro VALUES (5, '9780131857254', 'Agile Principles in C#',
2006, 9, 4);
INSERT INTO libro VALUES (6, '9789561124571', 'Desolacion',
1922, 7, 2);
INSERT INTO libro VALUES (7, '9788420437484', 'Rayuela',
1963, 5, 1);
INSERT INTO libro VALUES (8, '9788497593793', 'It',
1986, 2, 9);
INSERT INTO libro VALUES (9, '9788445000663', 'El Senor de los Anillos',
1954, 4, 6);
INSERT INTO libro VALUES (10, '9788499089515', 'Ficciones',
1944, 1, 7);
INSERT INTO libro VALUES (11, '9788498381498', 'El Principito',
1943, 7, 10);
INSERT INTO libro VALUES (12, '9788499926223', 'Sapiens',
2011, 4, 5);
INSERT INTO libro VALUES (13, '9780132350884', 'Clean Code',
2008, 9, 4);

-- LIBRO_AUTOR (14 filas; 'Agile Principles' tiene 2 autores y Robert
Martin tiene 2 libros -> N:M)
INSERT INTO libro_autor VALUES (1, 1, 1);
INSERT INTO libro_autor VALUES (2, 2, 2);
INSERT INTO libro_autor VALUES (3, 3, 3);
INSERT INTO libro_autor VALUES (4, 4, 4);
INSERT INTO libro_autor VALUES (5, 5, 5);
INSERT INTO libro_autor VALUES (6, 5, 6);
INSERT INTO libro_autor VALUES (7, 6, 7);
INSERT INTO libro_autor VALUES (8, 7, 8);
```

```
INSERT INTO libro_autor VALUES (9, 8, 9);
INSERT INTO libro_autor VALUES (10, 9, 10);
INSERT INTO libro_autor VALUES (11, 10, 11);
INSERT INTO libro_autor VALUES (12, 11, 12);
INSERT INTO libro_autor VALUES (13, 12, 13);
INSERT INTO libro_autor VALUES (14, 13, 5);

-- EJEMPLAR (ids 1..20). Los PRESTADO coinciden con los prestamos activos
(12 a 15)
INSERT INTO ejemplar VALUES (1, 1, 'PRESTADO',   'A-01');
INSERT INTO ejemplar VALUES (2, 1, 'DISPONIBLE', 'A-01');
INSERT INTO ejemplar VALUES (3, 2, 'PRESTADO',   'A-02');
INSERT INTO ejemplar VALUES (4, 3, 'DISPONIBLE', 'B-01');
INSERT INTO ejemplar VALUES (5, 4, 'DISPONIBLE', 'C-01');
INSERT INTO ejemplar VALUES (6, 4, 'DANADO',     'C-01');
INSERT INTO ejemplar VALUES (7, 5, 'DISPONIBLE', 'D-01');
INSERT INTO ejemplar VALUES (8, 6, 'DISPONIBLE', 'B-02');
INSERT INTO ejemplar VALUES (9, 7, 'DISPONIBLE', 'A-03');
INSERT INTO ejemplar VALUES (10, 8, 'DISPONIBLE', 'E-01');
INSERT INTO ejemplar VALUES (11, 9, 'PRESTADO',   'F-01');
INSERT INTO ejemplar VALUES (12, 9, 'DISPONIBLE', 'F-01');
INSERT INTO ejemplar VALUES (13, 10, 'DISPONIBLE', 'G-01');
INSERT INTO ejemplar VALUES (14, 11, 'DISPONIBLE', 'H-01');
INSERT INTO ejemplar VALUES (15, 12, 'DISPONIBLE', 'I-01');
INSERT INTO ejemplar VALUES (16, 13, 'PRESTADO',   'D-02');
INSERT INTO ejemplar VALUES (17, 13, 'DISPONIBLE', 'D-02');
INSERT INTO ejemplar VALUES (18, 2, 'PERDIDO',    'A-02');
INSERT INTO ejemplar VALUES (19, 7, 'DISPONIBLE', 'A-03');
INSERT INTO ejemplar VALUES (20, 11, 'DISPONIBLE', 'H-01');

-- SOCIO (ids 1..12)
INSERT INTO socio VALUES (1, '12345678-9', 'Camila',    'Rojas',
'camila.rojas@mail.com',    '912345678', 'Av. Principal 123');
INSERT INTO socio VALUES (2, '15678901-2', 'Diego',     'Soto',
'diego.soto@mail.com',      '923456789', 'Los Aromos 456');
INSERT INTO socio VALUES (3, '18901234-5', 'Valentina', 'Munoz',
'vale.munoz@mail.com',      '934567890', 'Pasaje Sur 78');
INSERT INTO socio VALUES (4, '20123456-7', 'Matias',    'Perez',
'matias.perez@mail.com',    '945678901', 'Calle Norte 910');
INSERT INTO socio VALUES (5, '16234567-8', 'Fernanda',  'Gonzalez',
'fer.gonzalez@mail.com',    '956789012', 'Los Olmos 234');
INSERT INTO socio VALUES (6, '17345678-K', 'Benjamin',  'Silva',
'benja.silva@mail.com',     '967890123', 'Av. Libertad 1500');
INSERT INTO socio VALUES (7, '19456789-1', 'Antonia',   'Contreras',
'anto.contreras@mail.com',  '978901234', 'Las Rosas 67');
```

```
INSERT INTO socio VALUES (8, '14567890-3', 'Tomas',     'Vargas',
'tomas.vargas@mail.com',    '989012345', 'Pje. Los Pinos 12');
INSERT INTO socio VALUES (9, '21678901-4', 'Josefa',    'Fuentes',
'josefa.fuentes@mail.com',  '990123456', 'Av. Central 880');
INSERT INTO socio VALUES (10, '13789012-6', 'Sebastian', 'Reyes',
'seba.reyes@mail.com',      '901234567', 'El Roble 345');
INSERT INTO socio VALUES (11, '22890123-5', 'Martina',   'Castro',
'martina.castro@mail.com',  '913579246', 'Los Canelos 91');
INSERT INTO socio VALUES (12, '11901234-7', 'Nicolas',   'Herrera',
'nico.herrera@mail.com',    '924681357', 'Av. Estacion 202');

-- EMPLEADO (ids 1..10)
INSERT INTO empleado VALUES (1, '11222333-4', 'Carolina', 'Diaz',
'Bibliotecaria Jefa');
INSERT INTO empleado VALUES (2, '13444555-6', 'Felipe',   'Lopez',
'Asistente');
INSERT INTO empleado VALUES (3, '14555666-7', 'Paula',    'Morales',
'Bibliotecaria');
INSERT INTO empleado VALUES (4, '15666777-8', 'Ignacio',  'Torres',
'Asistente');
INSERT INTO empleado VALUES (5, '16777888-9', 'Daniela',  'Araya',
'Bibliotecaria');
INSERT INTO empleado VALUES (6, '17888999-0', 'Cristobal','Navarro',
'Asistente');
INSERT INTO empleado VALUES (7, '18999000-1', 'Javiera',  'Espinoza',
'Encargada de Catalogo');
INSERT INTO empleado VALUES (8, '12111222-3', 'Rodrigo',
'Sepulveda','Asistente');
INSERT INTO empleado VALUES (9, '19222333-K', 'Catalina', 'Pizarro',
'Bibliotecaria');
INSERT INTO empleado VALUES (10, '20333444-5', 'Joaquin',  'Carrasco',
'Asistente');

-- PRESTAMO (ids 1..15). Todos los prestamos son por 7 dias.
-- 1: devuelto a tiempo
INSERT INTO prestamo VALUES (1, 1,  4,  1, SYSDATE-30, SYSDATE-23,
SYSDATE-25);
-- 2 a 11: devueltos con atraso (cada uno tiene multa)
INSERT INTO prestamo VALUES (2, 2,  5,  2, SYSDATE-60, SYSDATE-53,
SYSDATE-48); -- 5 dias
INSERT INTO prestamo VALUES (3, 3,  7,  3, SYSDATE-58, SYSDATE-51,
SYSDATE-49); -- 2 dias
INSERT INTO prestamo VALUES (4, 4,  9,  1, SYSDATE-55, SYSDATE-48,
SYSDATE-40); -- 8 dias
INSERT INTO prestamo VALUES (5, 5,  10, 4, SYSDATE-50, SYSDATE-43,
SYSDATE-42); -- 1 dia
```

```
INSERT INTO prestamo VALUES (6, 6,  13, 5, SYSDATE-47, SYSDATE-40,
SYSDATE-30); -- 10 dias
INSERT INTO prestamo VALUES (7, 7,  14, 2, SYSDATE-45, SYSDATE-38,
SYSDATE-35); -- 3 dias
INSERT INTO prestamo VALUES (8, 8,  15, 6, SYSDATE-40, SYSDATE-33,
SYSDATE-27); -- 6 dias
INSERT INTO prestamo VALUES (9, 9,  8,  7, SYSDATE-35, SYSDATE-28,
SYSDATE-24); -- 4 dias
INSERT INTO prestamo VALUES (10, 10, 12, 8, SYSDATE-30, SYSDATE-23,
SYSDATE-16); -- 7 dias
INSERT INTO prestamo VALUES (11, 11, 19, 9, SYSDATE-25, SYSDATE-18,
SYSDATE-16); -- 2 dias
-- 12 y 13: activos y atrasados (sin multa todavia -> para calcularla con
PL/SQL)
INSERT INTO prestamo VALUES (12, 12, 1,  10, SYSDATE-15, SYSDATE-8, NULL);
INSERT INTO prestamo VALUES (13, 3,  11, 1,  SYSDATE-12, SYSDATE-5, NULL);
-- 14 y 15: activos y al dia
INSERT INTO prestamo VALUES (14, 1,  3,  2, SYSDATE-3, SYSDATE+4, NULL);
INSERT INTO prestamo VALUES (15, 5,  16, 3, SYSDATE-1, SYSDATE+6, NULL);

-- MULTA (ids 1..10). Regla: $500 por dia de atraso
INSERT INTO multa VALUES (1, 2,  2500, 'S');
INSERT INTO multa VALUES (2, 3,  1000, 'S');
INSERT INTO multa VALUES (3, 4,  4000, 'N');
INSERT INTO multa VALUES (4, 5,  500,  'S');
INSERT INTO multa VALUES (5, 6,  5000, 'N');
INSERT INTO multa VALUES (6, 7,  1500, 'S');
INSERT INTO multa VALUES (7, 8,  3000, 'N');
INSERT INTO multa VALUES (8, 9,  2000, 'S');
INSERT INTO multa VALUES (9, 10, 3500, 'N');
INSERT INTO multa VALUES (10, 11, 1000, 'N');

-- RAZA (ids 1..10)
INSERT INTO raza VALUES (1,  'Mestizo');
INSERT INTO raza VALUES (2,  'Siames');
INSERT INTO raza VALUES (3,  'Persa');
INSERT INTO raza VALUES (4,  'Maine Coon');
INSERT INTO raza VALUES (5,  'Angora');
INSERT INTO raza VALUES (6,  'Bengali');
INSERT INTO raza VALUES (7,  'Ragdoll');
INSERT INTO raza VALUES (8,  'Azul Ruso');
INSERT INTO raza VALUES (9,  'British Shorthair');
INSERT INTO raza VALUES (10, 'Sphynx');

-- GATO (ids 1..12)
```

```
INSERT INTO gato VALUES (1,  'Nana',        'H', 'Atigrado Blanco',  DATE
'2019-03-12', DATE '2021-05-01', 1);
INSERT INTO gato VALUES (2,  'Trex',        'M', 'Blanco',           DATE
'2020-07-22', DATE '2021-05-01', 5);
INSERT INTO gato VALUES (3,  'Amira',       'H', 'Negro con Blanco', DATE
'2018-11-02', DATE '2021-08-15', 1);
INSERT INTO gato VALUES (4,  'Pandora',     'H', 'Carey Negro',      DATE
'2021-01-30', DATE '2022-02-10', 1);
INSERT INTO gato VALUES (5,  'Naruto',      'M', 'Blanco',           DATE
'2020-04-18', DATE '2022-06-20', 8);
INSERT INTO gato VALUES (6,  'Huachimingo', 'M', 'Negro',            DATE
'2022-09-05', DATE '2023-01-12', 1);
INSERT INTO gato VALUES (7,  'Molly',       'H', 'Americano',        DATE
'2021-06-14', DATE '2023-03-03', 1);
INSERT INTO gato VALUES (8,  'Canela',      'H', 'Tricolor',         DATE
'2022-02-27', DATE '2023-07-19', 2);
INSERT INTO gato VALUES (9,  'Morocha',     'H', 'Azul Ruso',        DATE
'2019-12-01', DATE '2023-10-08', 4);
INSERT INTO gato VALUES (10, 'Maki',        'H', 'Carey Gris',       DATE
'2023-04-10', DATE '2024-01-25', 3);
INSERT INTO gato VALUES (11, 'Cortazar',    'M', 'Blanco y negro',   DATE
'2022-08-19', DATE '2024-05-30', 9);
INSERT INTO gato VALUES (12, 'Jaspeao',     'H', 'Naranjo atigrado', DATE
'2024-02-14', DATE '2025-03-01', 6);

-- ALIMENTO (precio por porcion)
INSERT INTO alimento VALUES (1,  'Croquetas Adulto',    'SECO',   1000);
INSERT INTO alimento VALUES (2,  'Croquetas Gatito',    'SECO',   1200);
INSERT INTO alimento VALUES (3,  'Croquetas Light',     'SECO',   1100);
INSERT INTO alimento VALUES (4,  'Pate de Pollo',       'HUMEDO', 1500);
INSERT INTO alimento VALUES (5,  'Pate de Atun',        'HUMEDO', 1600);
INSERT INTO alimento VALUES (6,  'Sobre de Salmon',     'HUMEDO', 1800);
INSERT INTO alimento VALUES (7,  'Sobre de Pavo',       'HUMEDO', 1700);
INSERT INTO alimento VALUES (8,  'Snack de Atun',       'SNACK',  800);
INSERT INTO alimento VALUES (9,  'Barrita Cremosa',     'SNACK',  900);
INSERT INTO alimento VALUES (10, 'Galletas de Pescado', 'SNACK',  700);

-- ALIMENTACION (socio, gato, alimento, empleado, fecha, cantidad)
INSERT INTO alimentacion VALUES (1,  1,  1,  4,  1,  SYSDATE-20, 1);
INSERT INTO alimentacion VALUES (2,  2,  1,  8,  2,  SYSDATE-19, 2);
INSERT INTO alimentacion VALUES (3,  3,  2,  5,  3,  SYSDATE-18, 1);
INSERT INTO alimentacion VALUES (4,  4,  3,  1,  4,  SYSDATE-16, 1);
INSERT INTO alimentacion VALUES (5,  5,  4,  9,  5,  SYSDATE-15, 3);
INSERT INTO alimentacion VALUES (6,  6,  5,  6,  6,  SYSDATE-13, 1);
INSERT INTO alimentacion VALUES (7,  7,  6,  10, 7,  SYSDATE-12, 2);
INSERT INTO alimentacion VALUES (8,  8,  8,  2,  8,  SYSDATE-10, 1);
```

```
INSERT INTO alimentacion VALUES (9,  9,  9,  7,  9,  SYSDATE-9,  2);
INSERT INTO alimentacion VALUES (10, 10, 10, 8,  10, SYSDATE-7,  1);
INSERT INTO alimentacion VALUES (11, 11, 11, 3,  1,  SYSDATE-5,  1);
INSERT INTO alimentacion VALUES (12, 12, 12, 2,  2,  SYSDATE-4,  2);
INSERT INTO alimentacion VALUES (13, 1,  12, 5,  3,  SYSDATE-3,  1);
INSERT INTO alimentacion VALUES (14, 3,  7,  9,  4,  SYSDATE-2,  2);
INSERT INTO alimentacion VALUES (15, 1,  1,  6,  5,  SYSDATE-1,  1);

COMMIT;
```

7.1.3 Objetos de Apoyo

```
================================================================================
-- 02_objetos_apoyo.sql
-- Crea secuencias y tablas de auditoría necesarias para el Package y
Triggers
================================================================================

-- Secuencias para los PKs automáticos
CREATE SEQUENCE seq_prestamo START WITH 100 INCREMENT BY 1;
CREATE SEQUENCE seq_multa START WITH 100 INCREMENT BY 1;
CREATE SEQUENCE seq_alimentacion START WITH 100 INCREMENT BY 1;

-- Tabla de auditoría para el Trigger a nivel de sentencia
CREATE TABLE aud_gatoteca (
    id_aud NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tabla VARCHAR2(30),
    operacion VARCHAR2(10),
    usuario VARCHAR2(30) DEFAULT USER,
    fecha DATE DEFAULT SYSDATE
);
```

7.1.4 Package

```
-- ===================== ESPECIFICACIÓN ====================
CREATE OR REPLACE PACKAGE pkg_gatoteca AS

    -- Procedimientos CON parámetros
    PROCEDURE sp_registrar_prestamo (
        p_id_socio     IN prestamo.id_socio%TYPE,
        p_id_ejemplar  IN prestamo.id_ejemplar%TYPE,
        p_id_empleado  IN prestamo.id_empleado%TYPE
    );

    PROCEDURE sp_devolucion_ejemplar (
        p_id_prestamo  IN prestamo.id_prestamo%TYPE
    );

    PROCEDURE sp_registrar_alimentacion (
        p_id_socio            IN alimentacion.id_socio%TYPE,
        p_id_gato             IN alimentacion.id_gato%TYPE,
        p_id_alimento         IN alimentacion.id_alimento%TYPE,
        p_id_empleado         IN alimentacion.id_empleado%TYPE,
        p_cantidad_porciones  IN alimentacion.cantidad%TYPE
    );

    -- Función SIN parámetros
    FUNCTION fn_total_gatos_activos RETURN NUMBER;

    -- Función CON parámetros (usable desde SQL: solo lee, no modifica)
    FUNCTION fn_calcular_multa (
        p_id_prestamo IN prestamo.id_prestamo%TYPE
    ) RETURN NUMBER;

    -- Procedimiento SIN parámetros (procesamiento masivo)
    PROCEDURE sp_generar_multas_masivo;

END pkg_gatoteca;
/

-- ===================== CUERPO ====================
CREATE OR REPLACE PACKAGE BODY pkg_gatoteca AS

    -- ===== Constantes privadas =====
    c_dias_prestamo     CONSTANT NUMBER       := 7;
    c_min_porciones     CONSTANT NUMBER       := 1;
    c_max_porciones     CONSTANT NUMBER       := 5;
    c_multa_por_dia     CONSTANT NUMBER       := 500;
```

```
    c_estado_disponible CONSTANT VARCHAR2(20) := 'DISPONIBLE';

    -- Declaraciones anticipadas para las funciones usadas por varios
    -- procedimientos del cuerpo.
    FUNCTION fn_dias_atraso (p_id_prestamo NUMBER) RETURN NUMBER;
    FUNCTION fn_total_alimentacion (p_id_alimentacion NUMBER) RETURN
NUMBER;

    -- ===== Validaciones privadas =====
    FUNCTION existe_socio (p_id NUMBER) RETURN BOOLEAN IS
        v_cant NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cant FROM socio WHERE id_socio = p_id;
        RETURN v_cant > 0;
    END existe_socio;

    FUNCTION existe_empleado (p_id NUMBER) RETURN BOOLEAN IS
        v_cant NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cant FROM empleado WHERE id_empleado = p_id;
        RETURN v_cant > 0;
    END existe_empleado;

    FUNCTION existe_gato (p_id NUMBER) RETURN BOOLEAN IS
        v_cant NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cant FROM gato WHERE id_gato = p_id;
        RETURN v_cant > 0;
    END existe_gato;

    FUNCTION existe_alimento (p_id NUMBER) RETURN BOOLEAN IS
        v_cant NUMBER;
    BEGIN
        SELECT COUNT(*) INTO v_cant FROM alimento WHERE id_alimento = p_id;
        RETURN v_cant > 0;
    END existe_alimento;

    -- Devuelve el estado del ejemplar (NULL si no existe).
    -- Bloquea la fila para evitar que dos usuarios presten el mismo
ejemplar.
    FUNCTION estado_ejemplar (p_id NUMBER) RETURN VARCHAR2 IS
        v_estado ejemplar.estado%TYPE;
    BEGIN
        SELECT estado INTO v_estado
          FROM ejemplar
         WHERE id_ejemplar = p_id
```

```
            FOR UPDATE;
        RETURN v_estado;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN NULL;
    END estado_ejemplar;

    -- Días de atraso (negativo o 0 = a tiempo). Si no se ha devuelto, usa
hoy.
    FUNCTION fn_dias_atraso (p_id_prestamo NUMBER) RETURN NUMBER IS
        v_dias NUMBER;
    BEGIN
        SELECT NVL(TRUNC(fecha_devolucion_real), TRUNC(SYSDATE))
               - TRUNC(fecha_devolucion_estimada)
          INTO v_dias
          FROM prestamo
         WHERE id_prestamo = p_id_prestamo;
        RETURN v_dias;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 0;
    END fn_dias_atraso;

    -- Total a pagar por una alimentación (precio x porciones)
    FUNCTION fn_total_alimentacion (p_id_alimentacion NUMBER) RETURN NUMBER
IS
        v_total NUMBER;
    BEGIN
        SELECT ali.precio * ag.cantidad
          INTO v_total
          FROM alimentacion ag
          JOIN alimento ali ON ag.id_alimento = ali.id_alimento
         WHERE ag.id_alimentacion = p_id_alimentacion;
        RETURN v_total;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 0;
    END fn_total_alimentacion;

    -- Crea la multa del préstamo o actualiza su monto si sigue sin pagar.
    -- Si ya está pagada no la toca (multa tiene UNIQUE por préstamo, así
    -- que insertar otra provocaría ORA-00001). Usada por devolución y
masivo.
    PROCEDURE registrar_multa (p_id_prestamo NUMBER, p_monto NUMBER) IS
        v_pagada multa.pagada%TYPE;
    BEGIN
```

```
        IF p_monto IS NULL OR p_monto <= 0 THEN
            RETURN;
        END IF;

        SELECT pagada INTO v_pagada
          FROM multa
         WHERE id_prestamo = p_id_prestamo;

        IF v_pagada = 'N' THEN
            UPDATE multa
               SET monto = p_monto
             WHERE id_prestamo = p_id_prestamo;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            INSERT INTO multa (id_multa, id_prestamo, monto, pagada)
            VALUES (seq_multa.NEXTVAL, p_id_prestamo, p_monto, 'N');
    END registrar_multa;

    -- ===== Funciones públicas =====
    FUNCTION fn_total_gatos_activos RETURN NUMBER IS
        v_total NUMBER;
    BEGIN
        -- gato no tiene columna de estado: se cuentan todos los gatos
        SELECT COUNT(*) INTO v_total FROM gato;
        RETURN v_total;
    END fn_total_gatos_activos;

    FUNCTION fn_calcular_multa (
        p_id_prestamo IN prestamo.id_prestamo%TYPE
    ) RETURN NUMBER IS
    BEGIN
        IF p_id_prestamo IS NULL THEN
            RETURN 0;
        END IF;
        RETURN GREATEST(0, fn_dias_atraso(p_id_prestamo)) *
c_multa_por_dia;
    END fn_calcular_multa;

    -- ===== Préstamo =====
    PROCEDURE sp_registrar_prestamo (
        p_id_socio     IN prestamo.id_socio%TYPE,
        p_id_ejemplar  IN prestamo.id_ejemplar%TYPE,
        p_id_empleado  IN prestamo.id_empleado%TYPE
    ) IS
        v_estado      ejemplar.estado%TYPE;
```

```
        v_id_prestamo prestamo.id_prestamo%TYPE;
    BEGIN
        IF NOT existe_socio(p_id_socio) THEN
            RAISE_APPLICATION_ERROR(-20010, 'El socio ' || p_id_socio || '
no existe.');
        END IF;

        IF NOT existe_empleado(p_id_empleado) THEN
            RAISE_APPLICATION_ERROR(-20011, 'El empleado ' || p_id_empleado
|| ' no existe.');
        END IF;

        v_estado := estado_ejemplar(p_id_ejemplar);

        IF v_estado IS NULL THEN
            RAISE_APPLICATION_ERROR(-20012, 'El ejemplar ' || p_id_ejemplar
|| ' no existe.');
        ELSIF UPPER(v_estado) <> c_estado_disponible THEN
            RAISE_APPLICATION_ERROR(-20013, 'El ejemplar no está disponible
(estado: ' || v_estado || ').');
        END IF;

        -- El trigger trg_prestamo_estado_ejemplar marca el ejemplar como
PRESTADO
        INSERT INTO prestamo (id_prestamo, id_socio, id_ejemplar,
id_empleado,
                              fecha_prestamo, fecha_devolucion_estimada,
fecha_devolucion_real)
        VALUES (seq_prestamo.NEXTVAL, p_id_socio, p_id_ejemplar,
p_id_empleado,
                TRUNC(SYSDATE), TRUNC(SYSDATE) + c_dias_prestamo, NULL)
        RETURNING id_prestamo INTO v_id_prestamo;

        DBMS_OUTPUT.PUT_LINE('Préstamo ' || v_id_prestamo || ' registrado.
Devolver antes del '
                             || TO_CHAR(TRUNC(SYSDATE) + c_dias_prestamo,
'DD/MM/YYYY'));
    END sp_registrar_prestamo;

    -- ===== Devolución =====
    PROCEDURE sp_devolucion_ejemplar (
        p_id_prestamo  IN prestamo.id_prestamo%TYPE
    ) IS
        v_fecha_real    prestamo.fecha_devolucion_real%TYPE;
        v_dias_atraso   NUMBER;
        v_monto_multa   multa.monto%TYPE;
```

```
    BEGIN
        BEGIN
            SELECT fecha_devolucion_real
              INTO v_fecha_real
              FROM prestamo
             WHERE id_prestamo = p_id_prestamo
               FOR UPDATE;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                RAISE_APPLICATION_ERROR(-20020, 'El préstamo ' ||
p_id_prestamo || ' no existe.');
        END;

        IF v_fecha_real IS NOT NULL THEN
            RAISE_APPLICATION_ERROR(-20021, 'Este préstamo ya fue
devuelto.');
        END IF;

        -- El trigger trg_prestamo_estado_ejemplar deja el ejemplar
DISPONIBLE
        UPDATE prestamo
           SET fecha_devolucion_real = TRUNC(SYSDATE)
         WHERE id_prestamo = p_id_prestamo;

        v_dias_atraso := fn_dias_atraso(p_id_prestamo);

        IF v_dias_atraso > 0 THEN
            v_monto_multa := fn_calcular_multa(p_id_prestamo);
            registrar_multa(p_id_prestamo, v_monto_multa);

            DBMS_OUTPUT.PUT_LINE('Devolución con ' || v_dias_atraso || '
días de atraso. Multa: $' || v_monto_multa);
        ELSE
            DBMS_OUTPUT.PUT_LINE('Devolución registrada a tiempo, sin
multa.');
        END IF;
    END sp_devolucion_ejemplar;

    -- ===== Alimentación =====
    PROCEDURE sp_registrar_alimentacion (
        p_id_socio            IN alimentacion.id_socio%TYPE,
        p_id_gato             IN alimentacion.id_gato%TYPE,
        p_id_alimento         IN alimentacion.id_alimento%TYPE,
        p_id_empleado         IN alimentacion.id_empleado%TYPE,
        p_cantidad_porciones  IN alimentacion.cantidad%TYPE
    ) IS
```

```
        v_id_alimentacion alimentacion.id_alimentacion%TYPE;
        v_monto           NUMBER;
    BEGIN
        IF p_cantidad_porciones IS NULL OR p_cantidad_porciones <
c_min_porciones THEN
            RAISE_APPLICATION_ERROR(-20002, 'Debe comprar al menos ' ||
c_min_porciones || ' porción.');
        ELSIF p_cantidad_porciones > c_max_porciones THEN
            RAISE_APPLICATION_ERROR(-20001, 'La cantidad de porciones
excede el límite permitido por visita (Máx: ' || c_max_porciones || ')');
        END IF;

        IF NOT existe_socio(p_id_socio) THEN
            RAISE_APPLICATION_ERROR(-20003, 'El socio ' || p_id_socio || '
no existe.');
        END IF;

        IF NOT existe_gato(p_id_gato) THEN
            RAISE_APPLICATION_ERROR(-20004, 'El gato ' || p_id_gato || ' no
existe.');
        END IF;

        IF NOT existe_alimento(p_id_alimento) THEN
            RAISE_APPLICATION_ERROR(-20005, 'El alimento ' || p_id_alimento
|| ' no existe.');
        END IF;

        IF NOT existe_empleado(p_id_empleado) THEN
            RAISE_APPLICATION_ERROR(-20006, 'El empleado ' || p_id_empleado
|| ' no existe.');
        END IF;

        INSERT INTO alimentacion (id_alimentacion, id_socio, id_gato,
id_alimento,
                                  id_empleado, fecha, cantidad)
        VALUES (seq_alimentacion.NEXTVAL, p_id_socio, p_id_gato,
p_id_alimento,
                p_id_empleado, SYSDATE, p_cantidad_porciones)
        RETURNING id_alimentacion INTO v_id_alimentacion;

        v_monto := fn_total_alimentacion(v_id_alimentacion);

        DBMS_OUTPUT.PUT_LINE('Alimentación ' || v_id_alimentacion || '
registrada. Total: $' || v_monto);
    END sp_registrar_alimentacion;
```

```
    -- ===== Procesamiento masivo: multas de préstamos vencidos sin
devolver =====
    PROCEDURE sp_generar_multas_masivo IS
        v_monto       multa.monto%TYPE;
        v_procesados  NUMBER := 0;
    BEGIN
        FOR r IN (SELECT id_prestamo
                    FROM prestamo
                   WHERE fecha_devolucion_real IS NULL
                     AND TRUNC(fecha_devolucion_estimada) < TRUNC(SYSDATE))
LOOP

            v_monto := fn_calcular_multa(r.id_prestamo);
            registrar_multa(r.id_prestamo, v_monto);

            v_procesados := v_procesados + 1;
        END LOOP;

        DBMS_OUTPUT.PUT_LINE('Multas generadas/actualizadas: ' ||
v_procesados);
    END sp_generar_multas_masivo;

END pkg_gatoteca;
/

SHOW ERRORS PACKAGE BODY pkg_gatoteca
```

7.1.5 Triggers

```
-- ========================================================================
-- GatoTeca - TRIGGERS
```

```
-- 1) trg_prestamo_bir            FILA      BEFORE INSERT       prestamo
-- 2) trg_prestamo_estado_ejemplar FILA      AFTER INSERT/UPDATE prestamo
-- 3) trg_multa_aud_stmt          SENTENCIA AFTER INSERT/UPDATE/DELETE
multa

CREATE OR REPLACE TRIGGER trg_prestamo_bir
BEFORE INSERT ON prestamo
FOR EACH ROW
DECLARE
    v_estado ejemplar.estado%TYPE;
BEGIN
    SELECT estado INTO v_estado
      FROM ejemplar
     WHERE id_ejemplar = :NEW.id_ejemplar;

    IF UPPER(v_estado) <> 'DISPONIBLE' THEN
        RAISE_APPLICATION_ERROR(-20030,
            'Trigger: el ejemplar ' || :NEW.id_ejemplar || ' no está
disponible (estado: ' || v_estado || ').');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20032,
            'Trigger: el ejemplar ' || :NEW.id_ejemplar || ' no existe.');
END trg_prestamo_bir;
/

-- ---------- 2) FILA: mantener el estado del ejemplar ----------
-- Insert de préstamo      -> ejemplar PRESTADO
-- Se registra devolución  -> ejemplar DISPONIBLE
-- Solo modifica ejemplar (no consulta prestamo): no hay tabla mutante.
CREATE OR REPLACE TRIGGER trg_prestamo_estado_ejemplar
AFTER INSERT OR UPDATE OF fecha_devolucion_real ON prestamo
FOR EACH ROW
BEGIN
    IF INSERTING AND :NEW.fecha_devolucion_real IS NULL THEN
        UPDATE ejemplar
           SET estado = 'PRESTADO'
         WHERE id_ejemplar = :NEW.id_ejemplar;

    ELSIF UPDATING
          AND :OLD.fecha_devolucion_real IS NULL
          AND :NEW.fecha_devolucion_real IS NOT NULL THEN
        UPDATE ejemplar
           SET estado = 'DISPONIBLE'
```

```
         WHERE id_ejemplar = :NEW.id_ejemplar;
    END IF;
END trg_prestamo_estado_ejemplar;
/

-- ---------- 3) SENTENCIA: auditoría de multas ----------
-- Se ejecuta UNA vez por sentencia DML, sin importar cuántas filas
-- afecte (un UPDATE de 10 multas genera 1 registro, no 10).
CREATE OR REPLACE TRIGGER trg_multa_aud_stmt
AFTER INSERT OR UPDATE OR DELETE ON multa
DECLARE
    v_operacion VARCHAR2(10);
BEGIN
    IF INSERTING THEN
        v_operacion := 'INSERT';
    ELSIF UPDATING THEN
        v_operacion := 'UPDATE';
    ELSE
        v_operacion := 'DELETE';
    END IF;

    INSERT INTO aud_gatoteca (tabla, operacion)
    VALUES ('MULTA', v_operacion);
END trg_multa_aud_stmt;
/

SHOW ERRORS
```

7.1.6 Pruebas

==========

```
-- Pruebas de package + triggers con los datos de 01_gatoteca_init.sql
================================================================
SET SERVEROUTPUT ON

-- ---- P1: préstamo normal -> el trigger deja el ejemplar PRESTADO ----
-- ESPERADO: ejemplar 2 pasa de DISPONIBLE a PRESTADO
SELECT id_ejemplar, estado FROM ejemplar WHERE id_ejemplar = 2;
EXEC pkg_gatoteca.sp_registrar_prestamo(1, 2, 1);
SELECT id_ejemplar, estado FROM ejemplar WHERE id_ejemplar = 2;

-- ---- P2: prestar un ejemplar ya prestado ----
-- ESPERADO: ORA-20013 (package) y, por INSERT directo, ORA-20030 (trigger)
EXEC pkg_gatoteca.sp_registrar_prestamo(2, 2, 1);
INSERT INTO prestamo (id_prestamo, id_socio, id_ejemplar, id_empleado,
                      fecha_prestamo, fecha_devolucion_estimada)
VALUES (999, 2, 2, 1, SYSDATE, SYSDATE + 7);

-- ---- P3: funciones desde SQL ----
-- ESPERADO: 12 gatos; multa del préstamo 12 = 4000 (8 días) y del 13 =
2500 (5 días)
SELECT pkg_gatoteca.fn_total_gatos_activos AS total_gatos FROM dual;
SELECT id_prestamo, pkg_gatoteca.fn_calcular_multa(id_prestamo) AS
multa_calculada
  FROM prestamo
 WHERE id_prestamo IN (12, 13);

-- ---- P4: la función reproduce las multas ya existentes ----
-- ESPERADO: 0 filas (la multa calculada coincide con multa.monto en los
préstamos 2 a 11)
SELECT m.id_prestamo, m.monto,
pkg_gatoteca.fn_calcular_multa(m.id_prestamo) AS calculada
  FROM multa m
 WHERE m.monto <> pkg_gatoteca.fn_calcular_multa(m.id_prestamo);

-- ---- P5: procedimiento masivo sin parámetros (y repetido) ----
-- ESPERADO: "Multas generadas/actualizadas: 2" las dos veces; en multa
quedan
-- solo 2 filas nuevas (préstamos 12 y 13), sin duplicados (idempotente)
EXEC pkg_gatoteca.sp_generar_multas_masivo;
EXEC pkg_gatoteca.sp_generar_multas_masivo;
SELECT id_multa, id_prestamo, monto, pagada FROM multa WHERE id_prestamo IN
(12, 13);

-- ---- P6: devolución con atraso -> libera el ejemplar ----
-- ESPERADO: "8 días de atraso. Multa: $4000"; ejemplar 1 pasa a
DISPONIBLE;
```

```
-- la multa del préstamo 12 sigue siendo una sola fila
EXEC pkg_gatoteca.sp_devolucion_ejemplar(12);
SELECT id_ejemplar, estado FROM ejemplar WHERE id_ejemplar = 1;
SELECT COUNT(*) AS multas_prestamo_12 FROM multa WHERE id_prestamo = 12;

-- ---- P7: devolver dos veces ----
-- ESPERADO: ORA-20021
EXEC pkg_gatoteca.sp_devolucion_ejemplar(12);

-- ---- P8: trigger de sentencia vs filas afectadas ----
-- ESPERADO: aud_gatoteca sube en 1 aunque el UPDATE afecte varias multas
SELECT COUNT(*) AS aud_antes FROM aud_gatoteca;
UPDATE multa SET pagada = pagada WHERE pagada = 'N';
SELECT COUNT(*) AS aud_despues FROM aud_gatoteca;
SELECT tabla, operacion, usuario, fecha FROM aud_gatoteca ORDER BY id_aud;

-- ---- P9: alimentación, límites y caso normal ----
-- ESPERADO: ORA-20002 (0), ORA-20001 (6), ORA-20003 (socio 99999);
-- y la última: "Total: $2000"
EXEC pkg_gatoteca.sp_registrar_alimentacion(1, 1, 1, 1, 0);
EXEC pkg_gatoteca.sp_registrar_alimentacion(1, 1, 1, 1, 6);
EXEC pkg_gatoteca.sp_registrar_alimentacion(99999, 1, 1, 1, 2);
EXEC pkg_gatoteca.sp_registrar_alimentacion(1, 1, 1, 1, 2);

ROLLBACK;
```

7.2 Modelo MER

